Evaluating and Improving Adversarial Robustness of Machine Learning-Based Network Intrusion Detectors

Dongqi Han, Zhiliang Wang, *Member, IEEE*, Ying Zhong, Wenqi Chen, Jiahai Yang, *Senior Member, IEEE*, Shuqiang Lu, Xingang Shi, *Member, IEEE*, and Xia Yin, *Senior Member, IEEE*

***Abstract*— Machine learning (ML), especially deep learning (DL) techniques have been increasingly used in anomaly-based network intrusion detection systems (NIDS). However, ML/DL has shown to be extremely vulnerable to adversarial attacks, especially in such security-sensitive systems. Many adversarial attacks have been proposed to evaluate the robustness of ML-based NIDSs. Unfortunately, existing attacks mostly focused on *feature-space* and/or white-box attacks, which make impractical assumptions in real-world scenarios, leaving the study on practical gray/black-box attacks largely unexplored. To bridge this gap, we conduct the first systematic study of the gray/black-box *traffic-space* adversarial attacks to evaluate the robustness of ML-based NIDSs. Our work outperforms previous ones in the following aspects: (i) *practical*—the proposed attack can automatically mutate original traffic with extremely limited knowledge and affordable overhead while preserving its functionality; (ii) *generic*—the proposed attack is effective for evaluating the robustness of various NIDSs using diverse ML/DL models and non-payload-based features; (iii) *explainable*—we propose an explanation method for the fragile robustness of ML-based NIDSs. Based on this, we also propose a defense scheme against adversarial attacks to improve system robustness. We extensively evaluate the robustness of various NIDSs using diverse feature sets and ML/DL models. Experimental results show our attack is effective (e.g., $>$97% evasion rate in half cases for `Kitsune`, a state-of-the-art NIDS) with affordable execution cost and the proposed defense method can effectively mitigate such attacks (evasion rate is reduced by $>$50% in most cases).**

***Index Terms*— Network anomaly detection, network intrusion detection systems, adversarial machine learning, machine learning security, evasion attack.**

Manuscript received January 1, 2021; revised March 21, 2021; accepted May 3, 2021. Date of publication June 7, 2021; date of current version July 16, 2021. This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFB1800200. *(Corresponding author: Zhiliang Wang.)*

Dongqi Han, Zhiliang Wang, Ying Zhong, Wenqi Chen, Jiahai Yang, and Xingang Shi are with the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China (e-mail: handq19@mails.tsinghua.edu.cn; wzl@cernet.edu.cn; zhongy18@mails.tsinghua.edu.cn; chenwq19@mails.tsinghua.edu.cn; yang@cernet.edu.cn; shixg@cernet.edu.cn).

Shuqiang Lu and Xia Yin are with the Department of Computer Science, Tsinghua University, Beijing 100084, China (e-mail: lusq18@mails.tsinghua.edu.cn; yxia@tsinghua.edu.cn).

This article has supplementary material provided by the authors and color versions of one or more figures available at https://doi.org/10.1109/JSAC.2021.3087242.

Digital Object Identifier 10.1109/JSAC.2021.3087242

I. Introduction

NETWORK intrusion detection systems (NIDS) play a critical role in detecting malicious activities in networks. Based on the detection mechanism, NIDSs can be generally classified into two types [1]. *Signature-based* NIDSs have traditionally been used to detect known malicious traffic, but are unable to detect new attacks or advanced variants. By contrast, *anomaly-based* NIDSs, with machine learning (ML) and recently deep learning (DL) techniques, are receiving more and more attention due to the generalization ability to detect new and unknown attacks [2]–[8].

While intrigued by the great promise and performance, ML and DL models have shown to be extremely vulnerable to *adversarial attacks*—well-designed small changes of input can induce significant changes in the output of ML models [9]–[11]. To evaluate the robustness of ML-based systems, the widely-used approach is constructing adversarial attacks to demonstrate the upper bound of the robustness [12]. Such attacks must sufficiently reflect the adversary's ability in practical settings, otherwise will be useless for real-world systems.

Several adversarial attacks have been proposed for ML-based systems in other domains such as computer vision [12]–[14], natural language processing [15], and malware detection [16]–[19]. However, these methods are ill-suited for network intrusion detection for two reasons. Firstly, such attacks are modifying feature vectors (we refer these to *feature-space* attacks) instead of real input space (namely raw traffic for NIDSs). This is because in other domains the feature mapping from input space to feature space is reversible or can be easily formalized as differentiable functions. In contrast, feature extraction from network traffic to features in NIDS is neither invertible nor differentiable. As a result, feature-space attacks for NIDS are impractical in practice since adversarial traffic is difficult to derive from features. Secondly, we need to ensure that there is no communication violation or compromise of maliciousness when modifying malicious traffic, which is not a problem for other non-security domains.

As for *traffic-space* attacks (i.e., directly modifying traffic) against ML-based NIDSs, existing studies can be divided into three categories. Firstly, evasion attacks on traditional NIDSs have been well-studied [20], [21], but they are only useful for signature-based NIDSs without learning models.

0733-8716 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

Secondly, several traffic obfuscation and mutation methods have been proposed for evading traffic analysis based detection [22]–[24]. However, they are simply mimicking benign traffic or randomly mutating instead of exploiting vulnerabilities of ML/DL models with adversarial machine learning techniques. As a result, such attacks are very costly and inefficient, but attackers always consider attacking overhead in practice. Thirdly, the only traffic-space attack leveraging adversarial ML is a white-box attack [25]. Specifically, it assumes the attacker has all knowledge of the targeted NIDS, including used features and the ML model's architecture and parameters. However, such information is usually inaccessible for attackers in real-world scenarios.

In summary, existing adversarial attacks (whether feature-space or traffic-space) on ML-based NIDSs failed to perform in practice due to their impractical assumptions and over-simplified traffic mutation. To bridge this gap, we face three aspects of challenges to conduct a practical study on the adversarial robustness of ML-based NIDSs:

- *Practicability:* How to perform a *functionality-preserving traffic-space* (under *irreversible feature extraction*) attack with extremely *limited knowledge* and *affordable overhead*?
- *Generality:* How to propose a generic framework effective for evaluating the robustness of NIDSs using various *ML models* and *features*?
- *Explainability:* How to interpret the fragility and then improve the robustness of ML-based NIDSs against such adversarial attacks?

In this paper, we propose a novel practical adversarial attack by formulating it as a bi-level optimization problem with a two-step solution, which firstly solves the so-called *adversarial features* with small potential traffic-space mutation overhead and secondly searches the best traffic variant with the closest feature-space distance to the adversarial features. (This addressed *affordable overhead* challenge).

In the first step, we extend the prior ideas of using Generative Adversarial Network (GAN) to treat the targeted ML model as a black box and search adversarial features located at the low-confidence region of the surrogate model. Based on the transferable property [26] of adversarial attacks, the attack effect can be transferable to other ML models (addressing *limited knowledge* and *model generality* challenges). In the second step, we propose a heuristic packet crafting framework to automatically mutate malicious traffic. Based on domain knowledge of feature extraction in ML-based NIDSs, we present several traffic mutation operators which can influence all summarized features without breaking malicious functionality (addressing *functionality-preserving* and *feature generality*).

To address *irreversible feature extraction* challenge, we propose an invertible abstract traffic representation called *meta-info vector*. To address the *explainability* challenge, we explain the fragility of ML-based NIDSs by quantifying the extent to which each feature is manipulated. Based on this, we propose a defense scheme to improve system robustness against such attacks by removing most vulnerable features.

Contributions: Our major contributions involve presenting a novel adversarial attack framework and extensively evaluate the robustness of ML-based NIDSs, followed by interpretation and improvement of robustness against such attacks. Specifically, they are elaborated as follows:

- We present the first practical traffic-space adversarial attack on ML-based NIDSs under gray and black box assumptions. Compared with prior adversarial attacks, our threat model provides a more practical assumption of the attacker's ability and considers the attacking overhead. Compared with traditional means of crafting test-time evasive traffic, our attacks are not simply mimicking normal traffic but leverage adversarial machine learning techniques to search features located at the decision boundary of ML models and can automatically search evasive traffic mutations and are adaptive for dealing with different malicious traffic.
- We propose an explanation method for the fragile robustness of ML-based NIDSs. Based on this, a defense scheme was proposed to improve system robustness against the proposed adversarial attacks.
- We extensively evaluate our attack and defense method on a state-of-the-art NIDS `Kitsune`, as well as various ML-based NIDSs including six typical ML classifiers (including statistical models and deep neural networks) and two kinds of feature sets (packet-level and flow-level). Significant insights behind the attack are also explored through analysis.

The rest of the paper is organized as follows: We summarize the related work in Section II and provide backgrounds and motivations in Section III. Section IV introduces the threat model and problem statement, as well as motivation and overview of our attack framework. Section V and Section VI elaborate two steps in our attack. The defense schemes are provided in Section VII. Experimental results and findings are shown in Section VIII. We make discussions on limitations and improvements in Section IX. Section X concludes this study.

II. Related Work

We primarily introduce related work from the perspective of adversarial attacks since the robustness evaluation and primary contribution of this work is (by) proposing practical attacks.

A. Evasion Attacks on NIDSs

Evasion attacks on (N)IDS itself have been extensively studied [27], [28]. There have been extensive studies on evading signature-based systems [20], [21], [29], [30] and traditional anomaly-based systems [31]–[35]. However, such evasion attacks are only useful for signature-based NIDSs without learning models.

B. Adversarial Attacks on (Other) ML-Based Systems

There have been several works on adversarial attacks against ML-based systems in other domains. Adversarial example in the domain of computer vision has been widely

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF ASSUMPTIONS IN ADVERSARIAL ATTACKS AGAINST ML-BASED NIDSS

Impractical Assumptions	FWA	FGA	FBA	TWA	PGA	PBA†
Unlimited traffic modification	●	●	●	●	○	○
Directly modify features	●	●	●	○	○	○
Knowledge on classifiers*	●	◐	○	◐	○	○
Knowledge on feature extractors	●	●	●	●	●	○

†FWA/FGA/FBA/TWA: ***F**eature/**T**raffic-space **W**hite/**G**ray/**B**lack box **A**ttack*
PGA/PBA (Ours): ***P**ractical (traffic-space) **G**ray/**B**lack box **A**ttack*
∗Under this assumption, ● means the attacker has full knowledge about the ML model (including parameters, outputs, etc.), ◐ can only acquire the output probabilities, and ○ neither has any knowledge nor can access to the ML model.

studied [12], [13], [36], [37]. Gradient Descent method [16] and Genetic Programming (GP) [17] were used for evading PDF malware classifier. GAN-based methods were used to attack ML-based malware detection [18] and real-time video classification [14]. PSO-based methods were used to attack speech recognition [38], text classification [39] and object detection [40]. [15] leveraged stochastic optimization method to evade text sentiment analysis. However, due to the specificity of network traffic and NIDS, these methods cannot be directly applied.

C. Adversarial Attacks on ML-Based NIDSs

We propose a terminology of Adversarial attacks on ML-based NIDSs, and list impractical assumptions of existing attacks in Table I. Their specific definitions and related work are as follows:

1) *Feature-Space Attacks:* Feature-space attacks on evading ML-based NIDSs assume that attackers can directly modify the feature vectors. According to the attacker's knowledge of the targeted NIDS, existing feature-space attacks can be divided into three categories:
   - *Feature-Space White-Box Attack (FWA):* FWA requires full knowledge of the targeted NIDS. In [41], four gradient-based adversarial example attacks were directly used to evade an MLP classifier. Likewise, adversarial examples were leveraged in [42] to evade `Kitsune`. Similar gradient-based methods were also used in [43] to attack NIDSs for IoT networks, and GAN-based NIDSs [44].
   - *Feature-Space Gray-Box Attack (FGA):* FGA requires the feedbacks of targeted classifier (without other knowledge of the classifier compared with FWA). In [45], a GAN-based architecture IDS-GAN was proposed to generate evasive features. In [46], a boundary-based method was proposed to evade DoS detection systems by perturbing features.
   - *Feature-Space Black-Box Attack (FBA):* FBA neither requires feedbacks nor any knowledges on the classifier. In [47], features were randomly modified to evade botnet detectors.

   However, feature-space attacks are impractical since feature extraction in ML-based NIDSs is always irreversible.
2) *Traffic-Space Attacks:* Existing attacks that directly change network traffic can be divided into three categories:

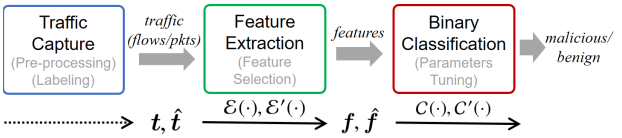


Fig. 1. The general pipeline of ML-based NIDSs.

   - *Traffic-Space White-Box Attack (TWA):* In [25], a white-box attack using similar mutation operators as ours was proposed. However, their assumption that the attacker has full knowledge of the NIDS is hard to achieve in practice.
   - *Random Mutations:* Several mutations were proposed in [22] to evade botnet detectors. Random obfuscations on traffic were proposed in [24]. However, these methods are purely stochastic and lack of theoretical guidance.
   - *Traffic Obfuscation:* Several traffic obfuscation methods such as [23] have been proposed for evading traffic analysis based detection. However, they were simply mimicking benign traffic and randomly mutation without exploiting vulnerabilities of ML/DL models. leveraging adversarial machine learning techniques.

III. BACKGROUND AND MOTIVATION

A. Targeted ML-Based NIDSs

In this study, we focused on evaluating the robustness of *ML-based NIDSs* using *non-payload-based* features. This kind of targeted systems is introduced as follows:

1) ML-Based NIDSs: In general, an ML-based NIDS consists of traffic capture, feature engineering, and classification as shown in Fig. 1. First, traffic is captured from the monitoring network. Then the feature set is extracted from raw traffic, selected, and finally fed into the ML classifier for training or detection. Note that, ML-based NIDSs need data pre-processing and manual feature extraction from raw traffic first, instead of relying solely on deep learning models for automatic feature learning. This is because network traffic is more sophisticated for having both textual and temporal characteristics, and its unstructured free text is difficult to be directly represented as variables.

2) Non-Payload-Based Features: In this study, we focus on evading ML-based NIDSs in which packets' payload is not inspected (called *non-payload-based*). We think this is reasonable due to two considerations: Firstly, we find that most ML-based NIDSs are likely to use *non-payload-based* features as inspecting payload is heavy and even impossible for encrypted traffic nowadays. Secondly, evading payload-based anomaly-based NIDSs has been well studied [34], [35]. Note that non-payload-based NIDSs prefer to detect attacks that rely on volume and/or iteration such as DoS/DDoS (Distributed Denial of Service), scanning, brute force, and Bot/Botnet. Other attacks related to specific content such as SQL injection are out of the scope of such NIDSs.

B. Adversarial Robustness Evaluation

1) Adversarial Attacks: Adversarial attacks on ML-based systems look for constrained perturbations to normal samples,

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

forcing the targeted model to misclassify the perturbed samples. In this study, we focus on one of the adversarial attacks called *evasion attacks*, which can force the targeted NIDS to misclassify malicious traffic variants as benign without loss of functionality of original malicious traffic.

2) Robustness Evaluation: In general, there are two approaches for evaluating the robustness of ML-based systems [12]. The first one is to theoretically prove the lower bound of the robustness through formal and mathematical methods. Although lower bound is more rigorous for security analysis, such methods are computationally intractable to verify practical systems [12]. By contrast, the second one, which constructs a reasonable adversarial attack and evaluates it on the targeted systems to demonstrate the upper bound of robustness (i.e., the robustness is no greater than whatever the attack can do if it succeeds), is widely used for practical systems.

C. Formulation and Limitations of Existing Attacks

For illustration purposes, we use the function $\mathcal{E}(\cdot)$ to represent the extraction from a series of related traffic to feature vectors and $\mathcal{C}(\cdot)$ to represent ML classifiers that take feature vectors as input and output the malicious probabilities. At any certain time, we denote by $\boldsymbol{t}$ and $\hat{\boldsymbol{t}}$ two series of related *original* and *mutated* malicious traffic used to extract two feature vectors $\boldsymbol{f}$ and $\hat{\boldsymbol{f}}$, respectively (i.e., $\mathcal{E}(\boldsymbol{t}) = \boldsymbol{f}$ and $\mathcal{E}(\hat{\boldsymbol{t}}) = \hat{\boldsymbol{f}}$).

As shown in Table I, we argue that existing attacks are under the following impractical assumptions:

1) *Feature-Space Mutation (in FWA/FGA/FBA):* Many previous studies merely find evasive features as solving $\texttt{argmin}_{\hat{\boldsymbol{f}}}\ \mathcal{C}(\hat{\boldsymbol{f}})$, which directly modify features' value without considering how to mutate traffic. However, features extraction is irreversible in ML-based NIDSs.
2) *White-Box Knowledge (in TWA/FWA/FGA):* Some work assumes that classifiers' specific output is attainable, then evasion attacks can be regarded as solving an optimization problem: $\texttt{argmin}_{\hat{\boldsymbol{t}}}\ \mathcal{C}\big(\mathcal{E}(\hat{\boldsymbol{t}})\big)$. However, NIDSs are always inaccessible for attackers in practice.
3) *Unlimited Mutation Overhead:* Existing attacks does not limit the attacker's overhead or ability to modify traffic (e.g., with respect to traffic volume or time delay). However, attackers always consider attack overhead in practice and the traffic is unlikely to be arbitrarily modified.

In light of this, we propose a more practical adversarial attack by relaxing feature-space and white-box assumption as well as adding the constraint of overhead.

IV. Attack Methodology

In this section, we first define the threat model in a more practical setting. Then, we provide the problem formulation of our attacks, followed by introduce several challenges and solutions to the problem. Finally, the overview of attack method is introduced. Some notations are listed in Table II.

A. Threat Model

We consider an attacker starts with a series of traffic with malicious intent and wants to evade a ML-based NIDS using non-payload-based features (i.e., *evasion attacks*). Unlike previous white/gray-box attacks, the attacker neither requires any knowledge about the target classifier nor its output label or probability. Unlike previous feature-space attacks, the attacker can only mutate original traffic generated from the devices he/she controls (i.e. *traffic-space* attack) at an affordable *overhead*. Additionally, based on the different attackers' knowledge of features in the targeted NIDS, two attacks can be performed:

- *Practical Gray-Box Attack (PGA):* This assumes features used in the targeted NIDS are known by the attacker. In other worlds, the attacker can build the same feature extractor as the targeted NIDS and use it to extract features exactly. This may seem extreme, but the features used in ML-based NIDSs are often published [4], [48]–[51].
- *Practical Black-Box Attack (PBA):* We assume a more practical case, in which the attacker has very limited or even no knowledge about the features used in the targeted NIDS. In this case, the attacker can only build a surrogate extractor based on the domain knowledge.

TABLE II
NOTATIONS IN PROBLEM FORMULATION

Notation	Description
$\boldsymbol{t}, \hat{\boldsymbol{t}}$	original and mutated malicious traffic
$\boldsymbol{f}, \hat{\boldsymbol{f}}$	feature vector extracted from original and mutated traffic
$\mathcal{E}(\cdot)$, $\mathcal{E}'(\cdot)$	targeted and surrogate feature extractor
$\mathcal{C}(\cdot)$, $\mathcal{C}'(\cdot)$	targeted and surrogate ML classifier
$\mathcal{L}(\cdot,\cdot)$	distance metric between two feature vectors

B. Practical Traffic-Space Evasion Attack Problem

According to the threat model, firstly, we relax two assumptions in Section III-C by training a substitute classifier $\mathcal{C}'(\cdot)$ with probabilistic output to approximate $\mathcal{C}(\cdot)$. This also solves the problem that some ML models without continuous output values (such as Isolation Forest) are difficult to optimize. Secondly, we build the surrogate feature extractor $\mathcal{E}'(\cdot)$. As for PGA, $\mathcal{E}'(\cdot)$ is exactly the same as $\mathcal{E}(\cdot)$, while is simulated as for PBA. How to build $\mathcal{C}'(\cdot)$ and $\mathcal{E}'(\cdot)$ will be introduced later. Additionally, we denote the mutation operation as $\mathcal{M}(\cdot)$ which can transform original traffic $\boldsymbol{t}$ to a set consisting of all possible mutated traffic $\hat{\boldsymbol{t}}$. We say a mutation operation is *safe* (denoted by $\mathcal{M}_s(\cdot)$) if the mutation can preserve the malicious functionality of $\boldsymbol{t}$. Besides, we also consider mutation overhead budget through an additional term to minimize the distance (denoted by $\mathcal{L}^*(\cdot,\cdot)$ with a weight coefficient λ) between $\hat{\boldsymbol{t}}$ and $\boldsymbol{t}$. Therefore, the attack problem can be formulated as solving:

$$\text{argmin}_{\hat{\boldsymbol{t}}}\ \ \mathcal{C}'\big(\mathcal{E}'(\hat{\boldsymbol{t}})\big) + \lambda\mathcal{L}^*(\hat{\boldsymbol{t}}, \boldsymbol{t}) \quad \text{s.t.} \quad \hat{\boldsymbol{t}} \in \mathcal{M}_s(\boldsymbol{t}). \tag{1}$$

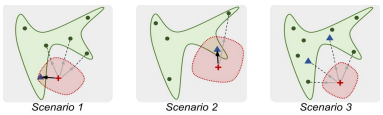
C. Challenges and Solutions

Problem (1) is hard to solve due to several challenges, such as the highly non-linear constraint, non-differentiable $\mathcal{E}'(\cdot)$, and hard-to-define $\mathcal{L}^*(\cdot,\cdot)$. Next, we will introduce how we transform the problem and solve it.

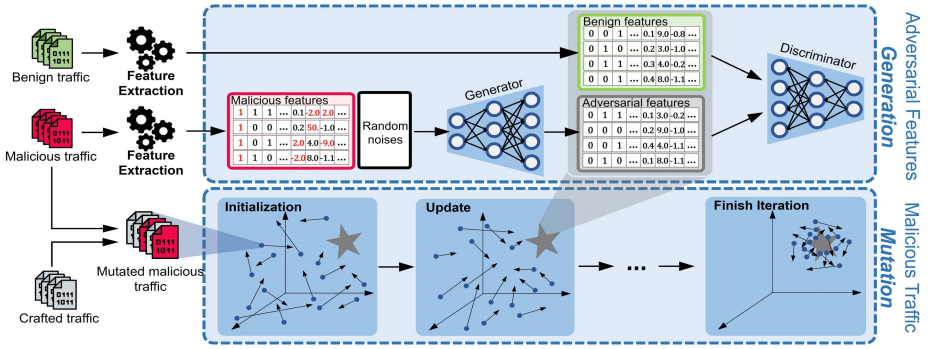
Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.



(a) Intuitive attack framework in feature space



(b) Motivation examples of adversarial features generation



(c) The complete framework of our evasion attack.

Fig. 2. Attack methodology. In (a) and (b), each plot depicts a high-dimensional feature space, in which the distribution of benign features in the targeted classifier is enclosed by a solid line with green; benign, malicious, and adversarial features are represented by small solid circles, crosses, and triangles respectively. In (b), the limited ability/overhead of an attacker is represented by a red neighborhood.

1) Bridging the Gap of Traffic-Feature Space: As mentioned before, we observe that the attacker's so-called ***overhead budget*** or ***ability*** to mutate traffic is always limited in practice. For one thing, an attacker is likely to have a *budget* of *overhead* (such as the extra time and crafted traffic volume to evade detection). For another, the attacker's *ability* to modify traffic is also limited. For examples, excessively increasing the interval time will cause the connection timeout, and injecting excessive traffic will occupy a lot of bandwidth and may be perceived by the victim. Considering the mutation overhead, directly searching the qualified $\hat{\boldsymbol{t}}$ from the sophisticated traffic space is inefficient due to the gap of traffic-feature space.

To address this problem, we introduce so-called ***adversarial feature*** denoted with $\boldsymbol{f}^\star$ to bridge this gap. We transform problem (1) into the following bi-level optimization problem:

$$\text{argmin}_{\hat{\boldsymbol{t}}} \quad \mathcal{L}\Big(\mathcal{E}'(\hat{\boldsymbol{t}}), \boldsymbol{f}^\star\Big) \tag{2}$$

$$\text{s.t. } \boldsymbol{f}^\star = \text{argmin}_{\boldsymbol{f}^\star} \ \mathcal{L}\Big(\boldsymbol{f}^\star, \mathcal{E}'(\boldsymbol{t})\Big) \tag{3}$$

$$\mathcal{C}'\big(\boldsymbol{f}^\star\big) < h \tag{4}$$

$$\hat{\boldsymbol{t}} \in \mathcal{M}_s(\boldsymbol{t}), \tag{5}$$

where $\mathcal{L}(\cdot,\cdot)$ is a distance metric between two feature vectors and h is the anomaly threshold for classification. Equation (4) means that $\boldsymbol{f}^\star$ is classified as benign. The high-level idea to solve the above problem is to separately solve the lower and upper-level objective function: We firstly solve adversarial feature $\boldsymbol{f}^\star$ in (3) under the constraint (4) in feature space, and then use the solved $\boldsymbol{f}^\star$ to search $\hat{\boldsymbol{t}}$ in (2) under the constraint (5). To give an intuition, Fig. 2a depicts these two steps from the perspective of feature space. Firstly, for each malicious feature, an $\boldsymbol{f}^\star$ is produced which can be not only classified as benign but also as close as possible to the malicious feature. Secondly, original malicious traffic is mutated to transfer its features to the closest adversarial(/benign) ones.

In general, $\boldsymbol{f}^\star$ lies on the low-confidence region of the classifier. Note that, $\boldsymbol{f}^\star$ is in feature space and does not need to correspond to traffic space. Our key observation is that *mutation overhead in traffic-space is correlated to the distance in feature-space* (Details and proof are in Appendix A). Thus, $\boldsymbol{f}^\star$ in feature space can reduce the overhead of mutating traffic. Fig. 2b also provides examples to demonstrate the necessity of $\boldsymbol{f}^\star$. Without guidance of adversarial features, a malicious feature may eventually fail to reach the nearest benign space (Scenario 1) or miss the transient benign space (Scenario 2).

2) Solving Adversarial Features $\boldsymbol{f}^\star$: Adversarial features generation needs to be: i) model-agnostic—we assume the attacker has no knowledge about the ML classifier; and ii) efficient—there may be tons of malicious/traffic in practice. Inspired by previous adversarial attacks [18], [45], GAN [52] is highly competent to generate adversarial features since i) the *discriminator* can be trained as a substitute for the targeted classifier (i.e., $\mathcal{C}'(\cdot)$ in (1) and (4)), thus can conduct model-agnostic attack; ii) once the *generator* is trained, it can generate $\boldsymbol{f}^\star$ efficiently for any malicious feature. Details will be introduced in Section V.

3) Solving Mutated Traffic $\hat{\boldsymbol{t}}$: Network traffic is unstructured free text thus cannot directly participate in the numerical calculation, so we vectorize traffic as high dimensional so-called *meta-info* vectors denoted with $\boldsymbol{x}$ involving meta information of packets' header. We propose several *safe* mutation operations on $\boldsymbol{x}$ to release the constrain (5). Therefore, we get the final unconstrained optimization problem:

$$\text{argmin}_{\boldsymbol{x}} \quad \mathcal{L}\Big(\mathcal{E}'\big(\mathcal{R}(\boldsymbol{x})\big), \boldsymbol{f}^\star\Big), \tag{6}$$

where $\mathcal{R}(\cdot)$ represents rebuild traffic from meta-info vectors, i.e., $\mathcal{R}(\boldsymbol{x}) = \hat{\boldsymbol{t}}$. However, most of dimensions in meta-info vectors are discrete and $\mathcal{E}'(\cdot)$ is non-differentiable, thus problem (2) is indeed a hard combinatorial optimization task (NP-complete). Hence, we resort to swarm intelligence algorithms to find approximate solutions. We employ PSO [53] due to its great adaptability on dealing with high dimensional and discrete tasks [38]–[40]. Details will be introduced in Section VI.

D. Overview of Attack Method

Now we briefly introduce the overview of our attack methodology from a more operational way. The workflow is illustrated in Fig. 2c, including the following two steps, corresponding to the aforementioned bi-level optimization:

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

- *Adversarial Features Generation:* We assume an attacker wants to launch some activities, which will induce a series of *malicious* traffic. First, the attacker needs to collect some *benign* traffic in the network he/she controls. Then, two kinds of traffic are extracted into features by the surrogate extractor, and fed into our GAN model. After the training phase, the generator is capable to generate adversarial features.
- *Malicious Traffic Mutation:* After generating adversarial features, we employ PSO with predefined *safe* operators to mutate malicious traffic automatically. Each particle in the swarm represents a vector consists of meta-info of mutated traffic. The swarm is iteratively searching the traffic-space under the guidance of the temporary best particle whose features are most similar to the adversarial feature. Finally, the best particle is selected after several iterations.

The details of the above two steps are elaborated in next two sections V and VI, respectively.

V. Generating Adversarial Features

We now introduce the procedure of generating adversarial features. Our enhanced GAN model is shown in Fig. 2c on the top, which consists of a generator and a discriminator.

A. Generator

The generator is a feed-forward neural network whose aim is to transform a malicious feature into its adversarial version. It takes the concatenation of a malicious feature vector $\boldsymbol{f}$ and a noise vector $\boldsymbol{z}$ from Gaussian distribution $p_{\boldsymbol{z}}(\boldsymbol{z})$ as input and outputs a generated feature vector represented by $G(\boldsymbol{f}, \boldsymbol{z})$. To train the generator, its loss function is defined as:

$$l_G = \mathbb{E}_{\boldsymbol{f}\in\mathbf{F}_{mal},\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log D(G(\boldsymbol{f},\boldsymbol{z})) + \mathcal{L}(\boldsymbol{f}, G(\boldsymbol{f},\boldsymbol{z}))], \tag{7}$$

where $\mathbf{F}_{mal}$ is the set of original malicious features. l_G should be minimized with respect to the weights in the generator's network. In this study, we extend prior GANs by additionally computing a construct error $\mathcal{L}(\cdot,\cdot)$ between the input and output. In this study, we use the root mean square error $\mathrm{RMSE}(\boldsymbol{x},\boldsymbol{x'}) = \sqrt{\sum_{i=1}^{n_d}(\boldsymbol{x}_i - \boldsymbol{x}'_i)^2 / n}$ where n_d is the dimensionality of the input and output features. Thus, the generated features can mimic the distribution of benign features while approaching malicious ones.

B. Discriminator

The discriminator trained as the surrogate classifier (i.e., $C'(\cdot)$) is used to distinguish generated features from benign ones. It is also a feed-forward neural network whose input consists of benign feature vectors and generated malicious feature vectors from the generator, and its output is a probability of determining an input vector is generated (also malicious). The discriminator is trained to maximize the output of generated input vector while minimize the output of benign input vectors. Thus, its loss function is:

$$l_D = -\mathbb{E}_{\boldsymbol{f}\in\mathbf{F}_{ben}}\log(1 - D(\boldsymbol{f})) - \mathbb{E}_{\boldsymbol{f}\in\mathbf{F}_{gen}}\log D(\boldsymbol{f}), \tag{8}$$

where $\mathbf{F}_{ben}$ is the set of features extracted from benign traffic collected in the network the attacker controls beforehand. $\mathbf{F}_{gen}$ is the set of features generated by the generator.

The training process is an iterative and mutual optimization between the generator and discriminator until a convergence. Then, the generated features from the generator can work as adversarial features $\mathbf{F}_{adver}$.

VI. Mutating Malicious Traffic

In this section, we introduce how to automatically mutate traffic through our PSO-based method. First, we design the mutation operators on malicious traffic. Then, we introduce the vectorization from traffic to meta-info vectors. Finally, we propose the PSO-based traffic mutation algorithm.

A. Basic Traffic Mutation Operators

To solve the evasive traffic mutation, we first design some basic parameterized *mutation operators*. The mutation operators should be able to affect as many types of features as possible to be generic, and also should be functionality-preserving (or safe, recall $\mathcal{M}_s(\cdot)$ in (5)) and stealthy to prevent being perceived by victims. However, it is challenging when attacker has limited or even no knowledge of the features used in the targeted system (i.e., PBA). To address this challenge, we propose a high-level summarization methodology of features used in non-payload-based NIDSs [4], [48]–[51] by dividing features extracted from network traffic into two dimensions—Temporal and Spatial: (More details are in Appendix B)

- Temporal features are related to the timing of traffic, such as the inter-arrival time of packets.
- Spatial features are further divided into Global and Local Spatial features:
  - Global Spatial features are the overall characteristics of traffic (for example, the volume of traffic, such as the number of bytes or the total number of packets).
  - Local Spatial features are related to the content of packets. Since we only focus on the packet headers and certain protocols, the types of Local Spatial features are limited.

Then, we design mutation operators which can affect all summarized high-level features (Fig. 7 in Appendix B provides an intuitive illustration) while preserving the functionality. Specifically, they consist of modifying original malicious traffic and injecting/adjusting crafted stealthy traffic:

Original Malicious Traffic Modification: We ensure that the original traffic will not be deleted and the order of packets will not be changed. Hence the only mutation operator is:

(a) Altering the **interarrival time** of packets in original traffic

Crafted Stealthy Traffic Injection: It is non-trivial to determine packer headers' content of crafted traffic. Firstly, we can only craft traffic sent from the attacker, and some fields (MAC/IP/port) in crafted packets need to be consistent with that of original packets nearby; otherwise the crafted packets cannot affect features extracted from original packets. Secondly, the assignment of other fields in header must meet

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CRAFTED TRAFFIC GENERATION METHOD

Traffic type	Generation methods
Any	Subtly assigning the **TTL** field so that the NIDS can receive the crafted packet but the victim cannot [25].
TCP	Requesting the establishment (i.e., send **SYN**) of an established or establishing connection again.
TCP (established)	Packets with smaller (already acknowledged) or larger **sequence number** than expected.
	Packets with smaller or larger **acknowledge number** than expected.
UDP/ICMP	Padding packets' payload with semantical-free content (such as randomly padding).
ICMP	Packets with deprecated type or code field.

the following requirements: 1) it will not compromise the maliciousness of the original traffic; 2) it will not cause the protocol semantics or communication violation (such as connection breakdown of TCP traffic); 3) it will not induce responses by the victim (for stealth and consistency of replay). In light of these requirements, we list optional methods for generating crafted traffic in Table III. We note that a previous method used in [25] by modifying TTL requires the knowledge of the victim's network topology, which is extremely strict. Hence, we propose other methods for different types of traffic without additional knowledge.

Crafted Traffic Adjustment: There are several adjustments for crafted packets after being injected:

(b1) Altering the **interarrival time** of packets in crafted traffic
(b2) Altering the **# protocol layer** of packets in crafted traffic
(b3) Altering the **payload size** of packets in crafted traffic

B. Meta-Information Vectorization

To facilitate numerical operations on structured traffic data, we *vectorize* traffic into *meta-info vectors* containing meta-information of original traffic. Note that unlike feature extraction, this vectorization is invertible, which means that it is easy to *rebuild* traffic from meta-info vectors. Meanwhile, the aforementioned mutation operators on traffic need to be reflected in the vectors. Details of the meta-info vectors and an illustrative example about vectorization and rebuilding between vectors and traffic are shown in Fig. 3, where $\boldsymbol{x}$ denotes the meta-info vectors.

To illustrate the meaning of each dimension in $\boldsymbol{x}$ and how they reflect the mutation operators, an $\boldsymbol{x}[i]$ is further divided into $\boldsymbol{x}_{mal}$ and $\boldsymbol{x}_{craft}[]$ used to represent a packet in original malicious traffic and several crafted packets right in front of it in time, respectively. The $\boldsymbol{x}_{mal}$ contains two parts: Timestamp corresponds to Mutation (a), and Number of crafted packets determines the size of the list $\boldsymbol{x}_{craft}$. Each craft packet denoted with $\boldsymbol{x}_{craft}[i]$ contains three parts: Interarrival time related to Mutation (b1) is the time interval from the previous packet; # protocol layers corresponding to Mutation (b2) refers to its number of layers in the TCP/IP protocol; Payload size directly reflects Mutation (b3).

1) Overhead Budget: In this study, we limit the attacker's overhead budget from two aspects. The first overhead denoted with l_c is the rate of the number of crafted packets to that of original packets. The second overhead denoted with l_t is the rate of time elapsed of mutated traffic to that of original traffic. In other words, the crafted packets number and time elapsed of mutated traffic must no more than l_c and l_t, respectively.

2) Dimensionality: Note that for the convenience of illustration, we split $\boldsymbol{x}$ into $\boldsymbol{x}_{mal}$ and $\boldsymbol{x}_{cal}$ with meaningful dimensions. For implementation, Meta-info vectors $\boldsymbol{x}$ are flattened into well-structured D-dimensional vectors. D can be computed according to the composition of $\boldsymbol{x}$:

$$D = N_{pkts}(3l_c + 2), \tag{9}$$

where N_{pkts} is the number of packets in malicious traffic.

C. PSO-Based Automatic Traffic Mutation

We now present our algorithm for automatically searching the best traffic mutants based on PSO (in (6)). The general framework of PSO is shown in Fig. 2c. Each particle represents a candidate traffic mutant, which consists of a *position* and *velocity* vector. Each position vector in the swarm is exactly the meta-info vector denoted by $\boldsymbol{x}^n$ ($n \in \{1, 2, \cdots, N_{swarm}\}$ where N_{swarm} is the number of particles in the swarm), and the velocity vector denoted by $\boldsymbol{v}^n$ shares the same shape with $\boldsymbol{x}^n$. Then we describe our PSO-based algorithm step by step. (We also provide the algorithm format in Appendix C.)

1) Initialization: At the very beginning, to sufficiently disperse initial particles in the search-space, fields in $\boldsymbol{x}_{craft}[]$ and # crafted pkts in $\boldsymbol{x}_{mal}$ are randomly initialized within the valid range. As for Timestamp of $\boldsymbol{x}_{mal}$, we divide the maximum interarrival time (related to l_t) between every two original packets into m equal parts, and Timestamp is randomly selected from these m-section points. And $\boldsymbol{v}^n$ is filled with 0 initially.

2) Effectiveness Evaluation: In each iteration, each particle is evaluated to judge how close they are to the objective according to its position $\boldsymbol{x}^n$ ($n \in \{1, 2, \cdots, N_{swarm}\}$). The effectiveness is defined as (10) according to (6):

$$Effectiveness(\boldsymbol{x}^n) = \mathbb{E}_{\boldsymbol{f}^\star \in \mathbf{F}_{adver}} \|\mathcal{E}'(\mathcal{R}(\boldsymbol{x}^n)) - \boldsymbol{f}^\star\|, \tag{10}$$

where can be viewed as three steps: First, mutated traffic is rebuilt (denoted by $\mathcal{R}$) from the position $\boldsymbol{x}^n$. Original traffic is directly retrieved after a replacement with Timestamp in $\boldsymbol{x}_{mal}$. As for crafted packets, after randomly determining their protocol type with reference to # protocol layers in $\boldsymbol{x}_{craft}[i]$, they can be rebuilt through the methods listed in VI-A. Fig. 3 also shows an example of traffic rebuilding. Second, mutated traffic is extracted (denoted by $\mathcal{E}'$) into features through the surrogate extractor. Third, the distance between extracted and adversarial features is computed as the effectiveness.

3) Update: After computing the effectiveness of $\boldsymbol{x}^n$, the position it has reached with the best effectiveness is recorded as its *individual best* position denoted by $\boldsymbol{b}^n$, and the best position in the swarm called *global best* position is denoted by $\boldsymbol{g}$. Then, each particle updates its velocity $\boldsymbol{v}^n$ as follows:

$$\boldsymbol{v}_d^n = \omega \boldsymbol{v}_d^n + r_1 c_1 (\boldsymbol{b}_d^n - \boldsymbol{x}_d^n) + r_2 c_2 (\boldsymbol{g}_d - \boldsymbol{x}_d^n), \tag{11}$$

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

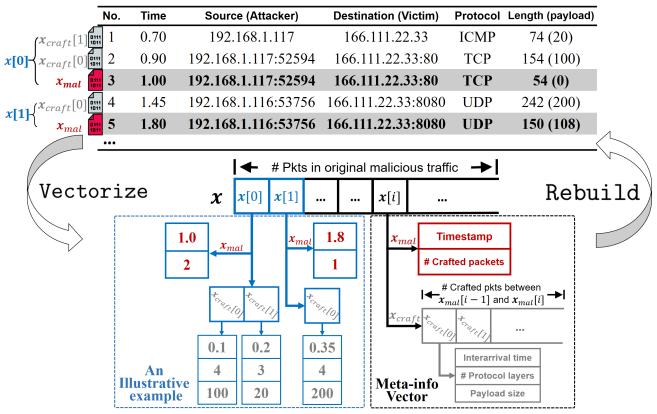


Fig. 3. Vectorization and rebuilding between vectors and traffic.

where ω, c_1, c_2 are weight coefficients of three terms and r_1, r_2 are random coefficients, and $d \in \{1, 2, \cdots, D\}$ (D is the same as (9)). And then position $\boldsymbol{x}^n$ is updated with the new $\boldsymbol{v}_d^n$:

$$\boldsymbol{x}_d^n = \texttt{clip}_d(\boldsymbol{x}_d^n + \boldsymbol{v}_d^n), \tag{12}$$

where the function `clip` limits the dimension beyond the value range to the minimum/maximum. Besides, discrete dimensions of $\boldsymbol{x}$ (e.g., # crafted pkts) are discretized by approximating them to the nearest discrete values.

4) Finish Iteration: The above two steps (evaluation and update) perform iteratively for a fixed number of iterations denoted by N_{iter}. Then, traffic rebuilt from the current global best position $\boldsymbol{g}$ becomes the best evasive mutant.

VII. Defense Scheme

To defend against the proposed attacks and improve the robustness of ML-based NIDSs, we introduce two probable methods in prior work, and then propose a novel defense scheme named *adversarial feature reduction*.

Adversarial Training [13]: This is a promising method widely used to defend against adversarial examples in the image domain by retraining the classifiers with correctly-labeled adversarial examples. However in our traffic-space attack, it can only reduce the attack effectiveness by limiting the generation of adversarial features.

Feature Selection [54]: This is an important step in feature engineering to remove redundant/irrelevant dimensions of features used in ML models, which can effectively improve detection performance and robustness.

Adversarial Feature Reduction: We propose a novel scheme to explain and defend against such traffic-space adversarial attacks. In a nutshell, we proactively simulate the proposed attack and then calculate the degree to which the value of each feature dimension in the mutated traffic is close to the adversarial features compared to original value (see Appendix D for details). The proximity rates of each feature dimension can be viewed as the adversarial robustness scores. Our main claim is the high dimensionality of features gives attackers an opportunity to exploit some vulnerable dimensions to evade detection. Hence, we propose an intuitive defense scheme by deleting partial feature dimensions with low robustness scores.

TABLE IV
Attack Traffic Datasets

Datasets	Attacks	# Test Pkts (Malicious)	# Training Pkts
Kitsune Dataset	Mirai Botnet	10,000 (8,079)	100,000 (50,000 benign & 50,000 malicious)
	Fuzzing	20,000 (14,898)	
	SSDP DoS	10,000 (7,987)	
CIC IDS2017	Port Scan	10,000 (2,569)	
	Brute Force	20,000 (6,136)	
	DDoS	10,000 (9,966)	

VIII. Experimental Evaluation

In this section, we first introduce the experimental settings[1] in VIII-A. In VIII-B and VIII-C, we compare the effectiveness of our attacks with baseline attacks. Several experiments are conducted using different kinds of malicious traffic (in VIII-B) and different NIDSs (in VIII-C). We evaluate the performance of our PBA attacks in VIII-D. Execution cost and impact of parameters are measured in VIII-E. We verify our attack is functionality-preserve in VIII-F. Finally, defense methods are evaluated in VIII-G.

A. Experimental Settings

1) Datasets: Table IV summarizes the information of traffic sets used in this study, including six well-known attacks from two up-to-date traffic datasets. *Kitsune Dataset* [4] was used to evaluate `Kitsune` by proactively performing a number of attacks in their video surveillance network. *CIC-IDS2017* [55] collected traffic for common attacks in a large-scale testbed, which covers all common devices and middleboxes. Note that, we additionally select different benign traffic with the same packet number of test set for adversarial feature generation.

2) Targeted NIDSs: Firstly, `Kitsune` [4] is evaluated as the state-of-the-art off-the-shelf NIDS, which consists of a packet-based feature extractor and Autoencoder model. Secondly, NIDSs using different ML classifiers (including traditional statistical learning [56] and deep neural networks) and different kinds of features are extensively evaluated (including models and extractors used in prior work on flow-level anomaly/intrusion detection [48], [57]):

- *Feature Extractors:* We evaluate two representative feature extractors: *AfterImage* [4] is a *packet-based* extractor in `Kitsune`. It computes incremental statistics of packet's size, count and jitter in various damped time windows. *CICFlowMeter* [50] is a *flow-based* extractor. It extracts several statistics (e.g., size, count, and duration) of connections.
- *ML Classifiers:* We apply six classifiers that are widely used in related work to comprehensively cover ML models [2], [3], [8]. *KitNET* is a deep, unsupervised, and ensemble Autoencoders used in `Kitsune`. *Multi-Layer Perceptron* (*MLP*) represents supervised deep learning models. *Logistics Regression* (*LR*), *Decision Tree* (*DT*)

[1]To strengthen scientific reproducibility, we provide a supplemental material consists of detailed settings of ML/DL models in our experiments, as well as the implementation code of adversarial packet crafting and optimized implementation of targeted NIDSs, which are available at: **https://github.com/dongtsi/TrafficManipulator**

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

TABLE V
NOTATIONS IN METRICS

Notation	Meaning
Pos	predicted positive number in original malicious traffic
$\widehat{\mathrm{Pos}}$	predicted positive number in mutated malicious traffic
$\widehat{\mathrm{Pos}}_{mal}$, $\widehat{\mathrm{Pos}}_{craft}$	malicious and crafted pkts' number in $\widehat{\mathrm{Pos}}$ ($\widehat{\mathrm{Pos}} = \widehat{\mathrm{Pos}}_{mal} + \widehat{\mathrm{Pos}}_{craft}$)
$\mathbf{F}_{adver}$	the set of adversarial features
$\mathbf{F}_{mal}$	the set of features extracted from original malicious traffic
$\hat{\mathbf{F}}_{mal}$	the set of features extracted from mutated malicious traffic
$\mathbb{E}(\cdot)$	mathematical expectation

and *Support Vector Machine* (*SVM*) represent supervised traditional ML models. *Isolation Forest* (*IF*) represents unsupervised anomaly detection model.

3) Baseline Attacks: For one thing, we find that previous attacks against signature-based NIDSs (like [20], [21]) and traditional anomaly-based NIDSs (like [34]) perform nearly no evasive effect. This is because these methods focused more on manipulating the payload. For another, feature-space attacks (e.g., [41], [42]) including FWA/FGA/FBA cannot participate either since they did not mutate the traffic. Hence, we existing employ traffic-space attacks as baselines:

- *Random Mutation:* Note that randomly mutating traffic is not a baseless weak attack, but appears in published works [22], [24]. We use two random mutation methods: **Random-ST** is randomly spreading interval-time between packets; **Random-Dup** is randomly duplicating partial original traffic. As for other methods, *packet injection* is not used since we find it has no effect on all traffic sets; *deleting/reordering packets* compromises the functionality.
- *Traffic-Space White-Box Attack (TWA):* The only work of TWA we find is [25], which uses similar mutation operators as ours. Since attackers have full knowledge of the targeted NIDSs in their assumption, the output probabilities are directly used as the optimization objective.

4) Metrics: We firstly present four new metrics with formulations and intuitive descriptions listed in Table VI. Notations used in the metrics are listed in Table V (Higher is better for all metrics). According to usage, evaluation metrics used in this work can be divided into three categories:

- *Evasive Effectiveness (MER, DER, and PDR): Original Malicious traffic Evasion Rate (MER)* and *Detection Evasion Rate (DER)* respectively reflect how much *original malicious traffic* and *all mutated traffic* (including crafted traffic) become evasive. That is to say, DER additionally considers whether the crafted traffic is classified as malicious, which can reflect whether our attack is stealthy. Since MER and DER are highly dependent on the anomaly threshold, we propose a more accurate metric by measuring the decline rate of the malicious probabilities outputted by the targeted classifier, namely *malicious Probability Decline Rate (PDR)*. The relationship between evasive effectiveness of attacks and robustness of NIDSs is: **For the same NIDS, attacks with better evasive effectiveness are more useful for evaluating robustness. For the same attack, NIDSs under better evasive effectiveness are less robust.**
- *Interpretable Indicator (MMR):* In order to explain and understand the reason and principle of evasion attacks on ML-based NIDSs, we propose an interpretable indicator *Malicious features Mimicry Rate (MMR)* which can explicitly show the change of features in the latent space during attacks. Specifically, MMR reflects the degree to which malicious features are close to adversarial features during the mutation.
- *Detection Performance:* We additionally use three typical metrics—*Precision*, *Recall*, and *F1-score*—to measure the detection performance of NIDSs.

B. Evasive Effectiveness of Different Attacks

In this section, we compare the evasive effectiveness of our PGA attacks with three baselines by evading `Kitsune` under different traffic sets. We also evaluate the effectiveness of adversarial features in our attacks by comparing our PSO-based algorithm with (GAN + PSO) and without (PSO) adversarial features. PSO only uses benign features (instead of adversarial features) when evaluating fitness (on line 6 of Alg. 1). We also compare the impact of overhead budget in our attacks using a lower ($l_c = 0.2, l_t = 2$) and a higher budget ($l_c = 0.5, l_t = 5$). Baseline attacks are all with the higher overhead budget. The results are shown in Fig. 4.

1) Evasive Effectiveness Comparison: As evident in the results of MER/PDR, our attack GAN + PSO performs very well relative to random mutations at the same budget ($l_c = 0.5, l_t = 5$). The effectiveness of random mutations is extremely unstable; each mutation only works under specific traffic sets. Moreover, our attack outperforms the state-of-the-art white-box attack (TWA), which also demonstrates the superiority of our attack method since the performance of white-box attacks is generally supposed to be better. As for DER, results show that the drop from MER to DER is $<3\%$ in most cases, which shows that the crafted traffic in our evasion attack is stealthy and unobservable even with $l_c = 0.5$ (more crafted packets).

2) Impact of Adversarial Features: We observe that using adversarial features indeed increases the evasive effectiveness (by 10-20% usually). Especially in *Fuzzing*, GAN + PSO increases MER/DER by more than 90% compared with PSO.

3) Impact of Overhead Budget: It is easy to understand that a higher overhead budget (namely, looser limitation) performs better results. Specifically, GAN + PSO with larger l_c and l_t have 20-30% higher MER/DER in most cases.

4) Performance of Different Traffic Sets: As shown in the results, our attack achieves $>97\%$ MER/DER on half of the traffic sets, as well as $>70\%$ MER/DER on five of six traffic sets. As for reasons of the relatively poor MER/DER in *DDoS*, this is because malicious features are originally farther from the benign space and beyond the attacker's ability/budget (See Scenario 3 in Figure 2b, if a malicious feature is beyond the attacker's overhead budget/ability, it cannot be transformed into a benign one by any means). In fact, we find the anomaly score (i.e, RMSE in `Kitsune`) of original features in *DDoS* is many orders of magnitude larger than other scenarios.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
EXPERIMENTAL METRICS

Metric Name	Formulation	Intuitive Description
Detection **E**vasion **R**ate (**DER**)	$1 - \frac{\widehat{\text{Pos}}}{\text{Pos}}$	% undetected mutated traffic (malicious and crafted) to originally detectable traffic.
original **M**alicious traffic **E**vasion **R**ate (**MER**)	$1 - \frac{\widehat{\text{Pos}_{mal}}}{\text{Pos}}$	% undetected mutated malicious traffic (exclude crafted) to originally detectable traffic.
malicious **P**robability **D**ecline **R**ate (**PDR**)	$1 - \mathbb{E}_{(\boldsymbol{f},\hat{\boldsymbol{f}})\in(\mathbf{F}_{mal},\hat{\mathbf{F}}_{mal})} \frac{C(\hat{\boldsymbol{f}})}{C(\boldsymbol{f})}$	To which extent the malicious probability output declines in the targeted ML classifier.
Malicious features **M**imicry **R**ate (**MMR**)	$1 - \mathbb{E}_{(\boldsymbol{f},\hat{\boldsymbol{f}},\boldsymbol{f}_a)\in(\mathbf{F}_{mal},\hat{\mathbf{F}}_{mal},\mathbf{F}_{adver})} \frac{\mathcal{L}(\hat{\boldsymbol{f}},\boldsymbol{f}_a)}{\mathcal{L}(\boldsymbol{f},\boldsymbol{f}_a)}$	To which extent features extracted from mutated traffic are close to the adversarial features.

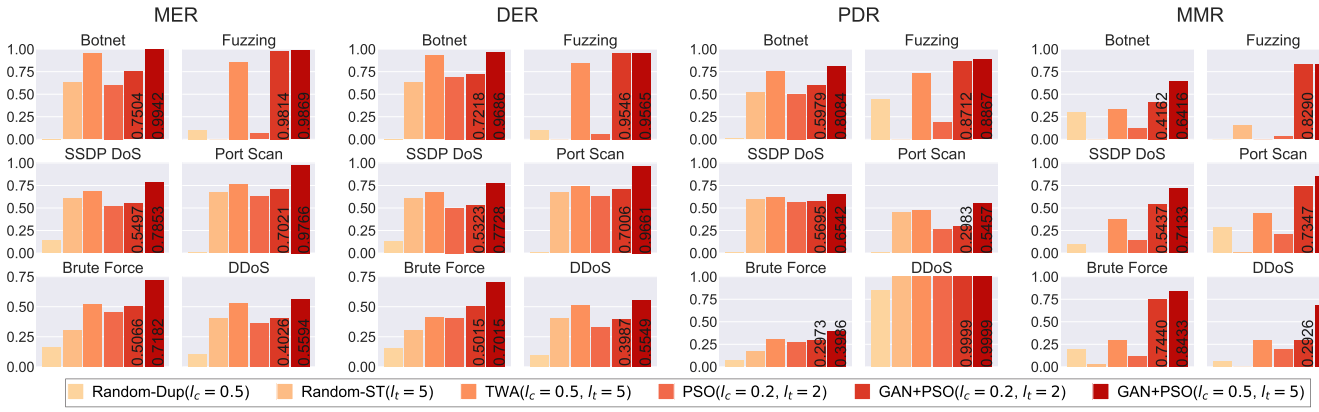


Fig. 4. The evasive effectiveness of our attacks compared with baselines (*higher is better from the perspective of attacks*).

TABLE VII
THE EVASIVE EFFECTIVENESS ON NIDSS WITH OTHER FEATURE EXTRACTORS AND ML CLASSIFIERS

(a) *Botnet*

Feature Extractor	ML Classifier	Detection			Evasive (**MER**)—*higher is better*			
		P	R	F1	R-Dup	R-ST	TWA	**Ours**
AfterImage	KitNET	0.98	0.92	0.95	0.20%	63.28%	94.98%	**99.42%**
	LR	0.96	0.90	0.93	0.67%	14.96%	50.17%	**54.74%**
	DT	0.79	0.90	0.84	0.61%	14.36%	49.13%	**60.36%**
	SVM	0.99	0.90	0.94	0.82%	9.10%	32.59%	**40.31%**
	MLP	0.96	0.97	0.97	0.87%	4.72%	10.59%	**45.15%**
	IF	0.95	0.93	0.94	0.76%	0.16%	0.52%	**33.63%**
CIC FlowMeter	KitNET	0.87	0.98	0.92	0.00%	9.69%	29.31%	**38.89%**
	LR	0.79	0.97	0.87	2.48%	1.87%	20.37%	**40.74%**
	DT	0.76	0.91	0.83	0.64%	3.70%	17.90%	**30.76%**
	SVM	0.78	0.98	0.87	0.00%	22.19%	41.35%	**84.62%**
	MLP	0.90	0.88	0.89	0.00%	0.00%	9.10%	**38.80%**
	IF	0.97	0.89	0.93	2.46%	0.00%	0.00%	**37.31%**

(b) *DDoS*

ML Classifier	Detection			Evasive (**MER**)—*higher is better*			
	P	R	F1	R-Dup	R-ST	TWA	**Ours**
KitNET	0.94	0.97	0.95	10.41%	13.74%	52.75%	**55.94%**
LR	0.96	0.91	0.93	27.09%	16.49%	64.90%	**70.59%**
DT	0.76	0.91	0.83	27.43%	17.43%	64.27%	**69.86%**
SVM	0.99	0.90	0.94	29.96%	18.20%	35.19%	**79.55%**
MLP	0.98	0.91	0.94	25.04%	9.58%	43.47%	**50.63%**
IF	0.85	0.89	0.87	0.00%	12.99%	0.0%	**17.71%**
KitNET	0.93	0.90	0.92	0.00%	8.77%	28.59%	**32.04%**
LR	0.70	0.73	0.71	1.25%	0.00%	14.80%	**36.82%**
DT	0.67	0.73	0.70	0.00%	3.01%	16.90%	**35.56%**
SVM	0.75	0.74	0.74	4.13%	2.04%	35.75%	**40.92%**
MLP	0.72	0.71	0.72	5.58%	15.45%	42.89%	**50.35%**
IF	1.00	0.89	0.94	0.00%	0.00%	13.44%	**25.98%**

This is exactly why its PDR is higher than others (over 99.99%) but MER/DER is lower. This finding also shows that it is necessary to consider attacker's ability/budget on mutating traffic as well as the original intensity of anomaly instead of purely comparing the evasion rate.

C. Robustness of Different Classifiers and Features

We conduct evasion attacks (our PGA and baselines with the higher budget) on different NIDSs described in Section VIII-A under *Botnet* and *DDoS* traffic (The other four traffic sets are not shown due to the reason of space, and using the current two sets is enough to draw the same conclusions). Since DER has been found to be very similar to MER, we use MER to measure the evasive performance, which is also the most concerning indicator for attackers. PDR is not used due to the inconsistency among ML models. Table VII lists the results.

1) Evasive Effectiveness Comparison: Compared with baseline attacks (each row in the table), our attack has broader generality for evading various ML classifiers using different kinds of features. Specifically, Random-RT always performs poorly while Random-Dup only has evasive effectiveness for a few cases. Once again, our attack outperforms TWA in all cases, especially for the Isolation Forest model. We attribute the generality to the feature-level mimicking in our model-agnostic attack. Meanwhile, it can be observed that the evasive performance of different classifiers is diverse significantly

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
THE EVASIVE EFFECTIVENESS OF OUR ATTACKS WITH LIMITED KNOWLEDGE OF FEATURES (PBAS) COMPARED WITH PGA

Attacks (Knowledge on features)	The Evasive Effectiveness (**MER / PDR**) on Different Traffic Sets—***higher is better***					
	Botnet	*Fuzzing*	*SSDP*	*Port Scan*	*Brute Force*	*DDoS*
PBA(0 %)	83.46% / 68.47%	82.68% / 70.04%	53.19% / 57.31%	35.56% / 26.32%	49.50% / 28.07%	33.06% / 99.99%
PBA(50%)	98.77% / 76.16%	98.64% / 82.38%	68.79% / 62.16%	72.72% / 49.78%	52.43% / 30.39%	41.22% / 99.99%
PBA(75%)	99.28% / 77.87%	98.26% / 81.89%	82.62% / 67.06%	76.82% / 52.38%	60.12% / 33.48%	50.45% / 99.99%
PGA(100%)	99.42% / 80.84%	98.69% / 88.67%	78.53% / 65.42%	97.66% / 54.57%	71.81% / 39.86%	55.94% / 99.99%

(each column of MER in the table). We think this is sound since the robustness of various models is different (just as their detection performance is also different).

2) Robustness of Different Feature Sets: NIDSs with flow-based features are slightly more robust against our attack as well as other attacks than packet-based ones. Note that, our mutation operators are packet-based (but not flow-based) due to the generic black/gray-box assumption. Per-packet mutation can attack flow-based NIDSs since flows consist of packets.

3) Robustness of Different Classifiers: Based on the results in ***Botnet***, we find that traditional ML methods are more robust than deep neural networks. Specifically, KitNET (in `Kitsune`) has the (almost) best detection performance but also suffers the highest evasion rate. In addition to the inherent vulnerability of neural networks [11], another probable reason is that KitNET clusters the features into groups, which gives attackers a better chance to influence more feature groups. Through experiments, we find the top 10% dimensions in original features exploited by our attack can eventually cover more than 50% of the features groups. The robustness of the different methods in ***DDoS*** is poor, while only IF still maintains good robustness. From the results of all cases, IF is relatively more robust, while the robustness of other ML models is unstable.

D. Attacks With Limited Knowledge of Features

So far, we have evaluated the effectiveness of our attack under the PGA assumption. We now extend our attack with limited knowledge of features used in targeted NIDSs (i.e., PBA). Specifically, we evaluate three types of attackers, who know the 75%, 50%, and 0% features that are accurately used by NIDSs, respectively. Recall Section IV-A, the only difference between PBA and PGA is the surrogate feature extractor used by the attacker. For PBA, besides limited known features, the attacker also extracts all commonly used features in Appendix B. Especially for PBA(0%), the attacker without any knowledge about the target system can only simulate the extractor by using other features. We use `Kitsune` as the targeted NIDS and evaluate the MER and PDR of three PBAs and the PGA in all traffic sets. The results are in Table VIII.

As mentioned before, we think that PDR can better reflect the evasive effectiveness compared with MER. As shown in the results, PBA(50%) and PBA(75%) perform high PDR that is similar to PGA. Even for attackers without any knowledge, PBA(0%) still has a strong evasive ability (Compared with PGA, the drop of PDR is within 20%). The key insight is that even if we cannot accurately know the features used by NIDSs, the mutation method computed by our attack through simulated features is also effective on real features. This finding seems to be very frustrating and frightening for such ML-based NIDSs, meaning that a weak attacker can easily make a considerable portion of malicious traffic become evasive.

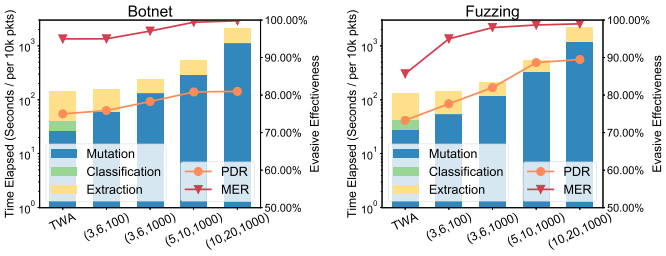


Fig. 5. Execution cost and impact of parameters.

E. Execution Cost and Impact of Parameters

It is necessary to measure the execution cost for attacks, especially for attackers with limited computing resources. Here, we use TWA as a comparison algorithm to represent the *lower bound* of execution time. This is because TWA requires the output value of the classifier, so the quality of candidate solutions (i.e., traffic mutants) can be quickly measured, but we do not have this knowledge in our attack. We also compare the impact of three key parameters on the execution time and evasive performance. We denote our attacks using different parameters with (N_{iter}, N_{swarm}, N_{adver}): N_{iter} and N_{swarm} denotes the number of iterations and particles in PSO, and N_{adver} denotes the number of adversarial features. The results are shown in Fig. 5, where we use `Kitsune` under two traffic sets since other sets have similar results.

It is shown that our attack with (3,6,100) can approximate the execution time of TWA while performing better evasive performance than TWA. For other parameters, our attacks are acceptable in execution time. Larger parameters have better evasive performance but will consume more time. To balance this trade-off, we think (5,10,1000) is the best combination of parameters, which is also chosen for other parts of experiments.

F. Verification of Malicious Functionality

To be rigorous, although we guarantee the mutation operators in Section VI-A will not compromise the malicious functionality of original traffic, we still verify the malicious functionality of the mutated traffic in all six traffic sets.

To measure the malicious functionality, we use three types of indicators: *attack effect*, *malicious behavior* and *attack efficiency*, and compare them in original and mutated traffic.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

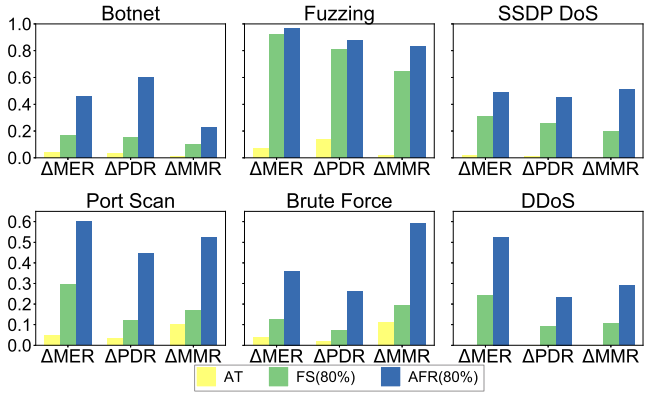


Fig. 6. The defense performance (*higher is better*).

Take *Botnet* as an example to illustrate the three indicators: In our selected traffic, an attacker used a malware called Mirai to scan IoT devices in the LAN and successfully scanned 8 open devices. In this scenario, *attack effect* is the final result of the attack, which is that 8 devices were successfully scanned. *Malicious behavior* contains all offensive behaviors regardless of whether they eventually affect, which is the number of scans. *Attack efficiency* is related to the elapsed time of the attack. Obviously, *attack effect* has a greater impact on functionality than *malicious behavior*, and the change rate of *attack efficiency* must be within the attacker's overhead budget l_t.

We use VMs and Dockers to simulate the experimental testbed for each traffic set by referring to their papers [4], [55]. Then we use Tcpreplay and Tcplivereplay to replay original and mutated attack traffic in the testbed and observe the three indicators. Since different attacks have diverse functionalities, the specific meanings of three indicators are case-by-case, making the validation experiment straightforward but tedious, so we put the details in Appendix E and leave the result here: *Mutated traffic generated through our evasion attack can preserve the malicious functionality*. Specifically, *attack effect* keeps unchanged in all cases and *malicious behavior* is reduced only in DoS/DDoS attacks (attack bandwidth is decreased due to increased time interval, but this reduction does not exceed the attacker's budget (l_t). As for *attack efficiency*, although our method may slow down some kinds of attack, the change rate of elapsed time is always $<l_t$.

G. Performance of Defense Schemes

In this section, we evaluate three defenses mentioned in Section VII. For adversarial training (AT), we retrain the ML classifiers with 80% relabeled adversarial features and use the remaining 20% for testing. For feature selection (FS), we use embedded Lasso regression model to retain 80% dimensions. As for our adversarial feature reduction (AFR), we also retain 80% feature dimensions. We use `Kitsune` in all traffic sets, and use the decline of metrics (ΔMER/PDR/MMR, also can be viewed as the improvement of robustness) to evaluate the defense performance. The results are shown in Fig. 6.

Compared with AT and FS, our AFR is very effective for improving the robustness of NIDSs by reducing MER/PDR/MMR. We observe that AT has a very limited and unstable defensive effect against the proposed attack. This is because it can limit the generation of adversarial features, but cannot prevent vulnerable feature dimensions from being exploited during traffic mutation. FS can perform better defense effectiveness in some cases. This shows that using fewer feature dimensions can increase the difficulty for attackers to transform the entire malicious features to some extent. We also measure the change of F1-score to evaluate whether the defense methods compromise the original detection performances. For reasons of spaces, they are not depicted since the change is quite small (within $\pm 5\%$).

IX. Discussions

We discuss limitations, considerations and potential improvements of our attacks as follows.

A. Limitations

As mentioned before, our method is designed for evading NIDSs without payload inspection, so it is invalid for systems additionally using payload-based detection. However, this problem can be easily solved by combining the *polymorphic blending attack* [34] with ours. And this can be easily implemented: leveraging polymorphic blending attack to encrypt the payload of original malicious traffic and using our method to inject crafted packets. Another limitation is that our attack is offline at present, but this can be solved by replaying mutated traffic since we have proofed that replayed traffic can conduct the same malicious intent as the original attack.

B. Why Black-Box Attacks

Compared with white-box attacks, black-box attacks are with higher feasibility since they need fewer prerequisites/requirements, especially in practical settings [58]. Note that, our scope in this work is from the perspective of attackers to propose more practical adversarial attacks and then propose the corresponding defense. Future work can focus on designing theoretically secure systems from the perspective of defense (according to Kerckhoff's principle [59]).

C. Background Traffic

In the proposed attack, we inject some crafted traffic which can be aggregated with original packets in order to impact features. However, some unpredictable background traffic (i.e., some traffic that is not controlled by the attacker but can also reach the victim or NIDS) may disrupt our mutated traffic on some features. Nonetheless, we find that only features aggregated by destination information (e.g., dstIP) are affected. Thus, the impact of background traffic is extremely limited (e.g., `Kitsune` has no features extracted only by destination).

D. Improving the Attack Effect

In this paper, we pay more attention to explore a more practical attack rather than try our best to improve the evasion rate. For one thing, we only use the default settings in the implementation of PSO and GAN in this study. For example,

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

we simply use the parameters of PSO algorithm recommended in [60]. For another, we use Euclidean distance to measure the similarity of features in this work. Future work should focus on whether other distance function or careful parameter tuning can perform better results.

X. Conclusion

This paper describes the first step toward developing a systematic study on practical traffic-space evasion attacks for adversarial robustness evaluation on ML-based NIDSs. Experimental results show our attack is effective (>97% evasion rate in half cases) and the proposed defense method can effectively mitigate such attacks. Our attack outperforms prior works while using affordable execution cost, and is effective even without any knowledge of the targeted systems. We extensively measure the robustness of various ML-based NIDSs and provide important findings. Our finding demonstrates that the paradigm of feature engineering should be shifted; we deem the detection performance together with anti-evasion robustness both need to be taken into consideration while designing feature sets. We firmly believe that our work provides important insights for improving the robustness of ML-based NIDSs and inspires more attention to the robust feature engineering in all ML-based systems.

Appendix A
Correlation Test Between Overhead of Feature-Space and Traffic-Space

Intuitively, it is easy to understand that the more difference between the two series of traffic, the more difference in their features' value. In turn, we speculate that the farther malicious traffic needs to move in the feature-space, the greater its overhead of mutation. To prove that, we conduct the following experiment to test the correlation between distance in feature-space and mutation overhead in traffic-space. The distance in feature-space is defined as Euclidean distance of two feature vectors, while mutation overhead in traffic-space is defined as the summation of delay overhead and crafted traffic volume overhead, which correspond to overhead budget l_t and l_c in our experiments respectively (Note that, we used actual overhead without budget limitation in this test). For illustration purposes, we use the notations $\mathcal{O}_t(\cdot)$ and $\mathcal{O}_c(\cdot)$ to represent the time elapsed and volume of a series of traffic, and still use $\mathcal{L}(\cdot,\cdot)$ to represent the Euclidean distance between two feature vectors, and $\mathcal{E}(\cdot)$ to represent feature extraction.

Correlation Test: The procedure of the correlation test is as follows:

1) We extract 10,000 features from the malicious traffic in *Botnet* traffic set denoted with $\boldsymbol{t}$ and call them *malicious features* denoted with $\boldsymbol{f}_{mal}$.
2) For each malicious feature, we randomly sample 10 *targeted features* denoted with $\boldsymbol{f}_{tar}$ at a distance range of $[100, 1000]$ (according to the distribution in our traffic sets) from the malicious feature. Thus, we have 100,000 sample points in total, each one consists of a *malicious feature* $\boldsymbol{f}_{mal}$ and a *targeted feature* $\boldsymbol{f}_{tar}$.
3) We execute the proposed PSO-based mutation algorithm (Alg. 2) for each $\boldsymbol{f}_{mal}$ and use its corresponding $\boldsymbol{f}_{tar}$ as the guide for traffic mutation (similar to adversarial feature in our attack). The mutated traffic solved after execution of the algorithm is denoted with $\hat{\boldsymbol{t}}$.
4) We keep *valid* sample points—$\mathcal{E}(\hat{\boldsymbol{t}})$ successfully approached the $\boldsymbol{f}_{tar}$ through the algorithm—for further test. Here we preserve sample points that satisfy $1 - \frac{\mathcal{L}(\mathcal{E}(\hat{\boldsymbol{t}}), \boldsymbol{f}_{tar})}{\mathcal{L}(\boldsymbol{f}_{mal}, \boldsymbol{f}_{tar})} > 0.7$.
5) For preserved sample points, we test the correlation between feature-space distance $\mathcal{L}(\boldsymbol{f}_{mal}, \boldsymbol{f}_{tar})$ and traffic-space overhead $(\frac{\mathcal{O}_t(\hat{\boldsymbol{t}})}{\mathcal{O}_t(\boldsymbol{t})} + \frac{\mathcal{O}_c(\hat{\boldsymbol{t}})}{\mathcal{O}_c(\boldsymbol{t})})$.

Results and Conclusion: Finally, we compute the Pearson Correlation Coefficient (PCC) of the preserved sample points, and the result is **0.8664** (strong correlation). Consequently, we draw the conclusion that adversarial features can efficiently reduce the overhead of mutating traffic by saving the distance of movement in feature-space.

Algorithm 1: PSO-Based Traffic Mutation Algorithm

Input: Hyperparameters $\omega, c_1, c_2, N_{iter}, N_{swarm}$ in PSO;
attacker's overhead budget l_c, l_t;
the original malicious traffic T;
Output: The mutated (/evasive) malicious traffic $\hat{\mathsf{T}}$.

for *each* $\boldsymbol{t}$ *in* T **do**
  $\boldsymbol{t}_v \leftarrow$Vectorize($\boldsymbol{t}$). ▷ *meta-info vectorization*
  $\mathsf{S} \leftarrow$Initialize($\boldsymbol{t}_v, N_{swarm}, l_c, l_t$). ▷ *initialize population*
  for $i = 1$ *to* N_{iter} **do**
    for $j = 1$ *to* N_{swarm} **do**
      $\mathsf{S}.\boldsymbol{d}_j \leftarrow \mathcal{L}\big(\mathcal{E}'(\text{Rebuild}(\mathsf{S}.\boldsymbol{x}_j)), \{\mathbf{F}_{adver}, \mathbf{F}_{ben}\}\big)$;
      $\mathsf{S}.\boldsymbol{y}_j \leftarrow \max(\mathsf{S}.\mathrm{y_j}, \mathsf{S}.\mathrm{d_j})$;
      $\mathsf{S}.\hat{\boldsymbol{y}} \leftarrow \max(\mathsf{S}.\hat{\mathrm{y}}, \mathsf{S}.\mathrm{y_j})$;
    end
    for $j = 1$ *to* N_{swarm} **do** ▷ *update each* $\boldsymbol{v}$ *and* $\boldsymbol{x}$
      $\boldsymbol{v}_{cog} \leftarrow \mathsf{S}.\boldsymbol{y}_j - \mathsf{S}.\boldsymbol{x}_j$; ▷ *cognitive force*
      $\boldsymbol{v}_{soc} \leftarrow \mathsf{S}.\hat{\boldsymbol{y}} - \mathsf{S}.\boldsymbol{x}_j$; ▷ *social force*
      Randomly sample $r_1, r_2 \sim p_{\mathrm{uniform}(0,1)}$.
      $\mathsf{S}.\boldsymbol{v}_j \leftarrow \omega\mathsf{S}.\boldsymbol{v}_j + r_1c_1\boldsymbol{v}_{cog} + r_2c_2\boldsymbol{v}_{soc}$;
      $\mathsf{S}.\boldsymbol{x}_j \leftarrow$ UpdateX($\mathsf{S}.\boldsymbol{x}_j, \mathsf{S}.\boldsymbol{v}_j, l_c, l_t$).
    end
  end
  Append $\hat{\boldsymbol{t}} \leftarrow$ Rebuild($\mathsf{S}.\hat{\boldsymbol{y}}$) into $\hat{\mathsf{T}}$.
end
return $\hat{\mathsf{T}}$

Appendix B
High-Level Feature Summarization and Basic Mutation Operators

As mentioned in Section VI-A, we propose a high-level summarization of features used in ML-based NIDSs, and then design basic mutation operators to affect high-level features. In this appendix, we demonstrate the versatility of our summarization method and how basic mutation operators affect all high-level features by using existing NIDSs as case studies.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

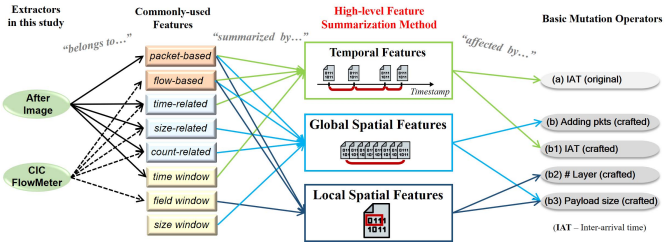


Fig. 7. Illustration of the versatility of our summarization method and how basic mutation operators affect all high-level features.

Algorithm 2: Adversarial Feature Evaluation Algorithm

Input: $\mathbf{F}_{mal}$, $\hat{\mathbf{F}}_{mal}$, $\mathbf{F}_{adver}$, $\mathbf{F}_{ben}$, anomaly threshold h;
Output: Adversarial feature score $\boldsymbol{s}$ of each dimension.
1 $n_d \leftarrow$ the dimensionality of a feature vector;
2 $n_f \leftarrow$ Number of features in $\mathbf{F}_{mal}$ or $\hat{\mathbf{F}}_{mal}$;
3 Initialize $\boldsymbol{s}$ with n_d zeros.
4 **for** *each* $\boldsymbol{f}, \hat{\boldsymbol{f}}$ *in* $\mathbf{F}_{mal}$,$\hat{\mathbf{F}}_{mal}$; $i = 0$ *to* $n_f - 1$ **do**
5 Initialize an $\boldsymbol{r}$ with n_d zeros.
6 **for** $j = 1$ *to* n_d **do** ▷ *each dimension*
7 $\boldsymbol{r}_j \leftarrow$MMR($\boldsymbol{f}_j, \hat{\boldsymbol{f}}_j, \mathbf{F}_{adver}, \mathbf{F}_{ben}$);
 ▽ *add a penalty if successfully evading*
8 **if** $\mathcal{E}(\boldsymbol{f}) > h$ *and* $\mathcal{E}(\hat{\boldsymbol{f}}) < h$ **then** $\boldsymbol{s}_j \leftarrow \boldsymbol{s}_j - \boldsymbol{r}_j$;
9 **else** $\boldsymbol{s}_j \leftarrow \boldsymbol{s}_j + (1 - \boldsymbol{r}_j)$; ▷ *add a reward*
10 **end**
11 **end**
12 Normalize $\boldsymbol{s}$ through dividing each dimension by n_f.
13 **return** $\boldsymbol{s}$

First, widely-used feature extraction in existing work [4], [48]–[51] can be generally divided into three steps:

1) *Choice of Data Form:* Network traffic is generally processed in two forms: *packets* and *sessions* (or flows). The difference between them is session-based extractions look at aggregated information of related packets.
2) *Choice of Basic Measurements:* Three widely-used measurements are *size* (e.g., packets' length), *count* (e.g., # packets), and *time* (e.g., inter-arrival time between packets).
3) *Process of Basic Measurements:* Given measurements in certain data forms, existing extraction methods prefer window-based collection (e.g., data with same source IP or within a fixed time interval) and/or statistics computing (e.g., mean and variance) to get advanced features.

Then in Fig. 7, we show how two extractors used in this work and commonly-used features be summarized by our summarization method, and how mutation operators affect all features.

APPENDIX C
PSO-BASED TRAFFIC MUTATION ALGORITHM

The PSO-based traffic mutation algorithm is shown in Algorithm 1. In each iteration, we firstly evaluate each particle's evasive effectiveness (on line 6) and update individual best and global best positions (on lines 7-8), which are respectively used to compute cognitive force (on line 11) and social force (on line 12). Then, each particle's $\boldsymbol{v}$ is updated by multiplying constant weights ω, c_1, and c_2 with inertia, cognitive, and social items, respectively (on line 14). Each particle's $\boldsymbol{x}$ is then updated according to $\boldsymbol{v}$ (on line 15).

TABLE IX
COMPARISON OF THE MALICIOUS FUNCTIONALITY

(a) *Botnet*

Indicators	Original	Mutated	Comparison
Number of open devices scanned	8	8	—
Total number of scans	8500	8500	—
Time elapsed	0.795s	2.364s	↑ (197%)

(b) *Fuzzing*

Indicators	Original	Mutated	Comparison
Impact of fuzzing on target systems	This cannot be simulated because we have no specific information about the targeted system		
# pkts containing fuzzing payload	5353	5353	—
Time elapsed	3.81s	2.90s	↓ (24%)

(c) *SSDP DoS*

Indicators	Original	Mutated	Comparison
Impact of DoS attack on targeted systems	This cannot be simulated because we have no specific information about the targeted system		
Attack bandwidth	35Mbps	20Mbps	↓ (39%)
Time elapsed	This can be reflected by the prevent indicator (bandwidth)		

(d) *Brute Force*

Indicators	Original	Mutated	Comparison
Whether the targeted FTP server is cracked	This cannot be simulated because we do not know the true password of the targeted system		
Total number of password attempts	60	60	—
Time elapsed	2.83s	11.26s	↑ (298%)

(e) *Port Scan*

Indicators	Original	Mutated	Comparison
Number of open ports scanned	3	3	—
Total number of scans	4810	4810	—
Time elapsed	6.55s	26.89s	↑ (310%)

(f) *DDoS*

Indicators	Original	Mutated	Comparison
Impact of DDoS attack on targeted systems	This cannot be simulated because we have no specific information about the targeted system		
Attack bandwidth	107Mbps	68Mbps	↓ (36%)
Time elapsed	This can be reflected by the prevent indicator (bandwidth)		

APPENDIX D
ADVERSARIAL FEATURE EVALUATION ALGORITHM

Algorithm 2 shows the specific robustness evaluation method mentioned in Section VII. we proactively simulate the evasion attack and measure the MMR (on line 7). Then by considering whether this feature vector can evade the classifier, a penalty or reward is added to adversarial robustness score (on lines 8-9). Finally, the adversarial robustness of a feature set is quantized into the score between -1 to 1 (on line 12) of each feature.

APPENDIX E
VERIFYING THE MALICIOUS FUNCTIONALITY

Detailed results of verifying malicious functionality of all six attack traffic sets are listed in Table IX. Note that, although the *attack effect* cannot be measured in some cases, in fact the results of *attack effect* are generally the same as *malicious*

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

behavior. For example, in *Brute Force*, we do not know the true password of the victim server, but if we guarantee that all the original password attempts exist in the mutated traffic, then obviously the final result is the same.

REFERENCES

[1] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, nos. 1–2, pp. 18–28, Feb. 2009.

[2] E. Hodo, X. Bellekens, A. Hamilton, C. Tachtatzis, and R. Atkinson, "Shallow and deep networks intrusion detection system: A taxonomy and survey," 2017, *arXiv:1701.02145*. [Online]. Available: http://arxiv.org/abs/1701.02145

[3] S. Gulghane, V. Shingate, S. Bondgulwar, G. Awari, and P. Sagar, "A survey on intrusion detection system using machine learning algorithms," in *Proc. Int. Conf. Innov. Data Commun. Technol. Appl.* Cham, Switzerland: Springer, 2019, pp. 670–675.

[4] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018, pp. 1–15.

[5] Y. Zhong *et al.*, "HELAD: A novel network anomaly detection model based on heterogeneous ensemble learning," *Comput. Netw.*, vol. 169, Mar. 2020, Art. no. 107049.

[6] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martinez-del-Rincon, and D. Siracusa, "Lucid: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Trans. Netw. Service Manage.*, vol. 17, no. 2, pp. 876–889, Jun. 2020.

[7] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020, doi: 10.1109/TIFS.2020.2991876.

[8] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, Feb. 2020, Art. no. 102419.

[9] M. Barreno, B. Nelson, A. D. Joseph, and J. D. Tygar, "The security of machine learning," *Mach. Learn.*, vol. 81, no. 2, pp. 121–148, 2010.

[10] J. D. Tygar, "Adversarial machine learning," *IEEE Internet Comput.*, vol. 15, no. 4, pp. 4–6, Sep./Oct. 2011.

[11] C. Szegedy *et al.*, "Intriguing properties of neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014, pp. 1–10.

[12] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 39–57.

[13] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, pp. 1–11.

[14] S. Li *et al.*, "Stealthy adversarial perturbations against real-time video classification systems," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2019, pp. 1–5.

[15] J. Li, S. Ji, T. Du, B. Li, and T. Wang, "TextBugger: Generating adversarial text against real-world applications," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2019, pp. 1–15.

[16] N. Rndic and P. Laskov, "Practical evasion of a learning-based classifier: A case study," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, May 2014, pp. 197–211.

[17] W. Xu, Y. Qi, and D. Evans, "Automatically evading classifiers," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2016, pp. 1–15.

[18] W. Hu and Y. Tan, "Generating adversarial malware examples for black-box attacks based on GAN," 2017, *arXiv:1702.05983*. [Online]. Available: http://arxiv.org/abs/1702.05983

[19] Y. Chen, S. Wang, D. She, and S. Jana, "On training robust PDF malware classifiers," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 2343–2360.

[20] M. Handley, V. Paxson, and C. Kreibich, "Network intrusion detection: Evasion, traffic normalization, and end-to-end protocol semantics," in *Proc. USENIX Secur. Symp.*, 2001, pp. 1–18.

[21] T.-H. Cheng, Y.-D. Lin, Y.-C. Lai, and P.-C. Lin, "Evasion techniques: Sneaking through your intrusion detection/prevention systems," *IEEE Commun. Surveys Tuts.*, vol. 14, no. 4, pp. 1011–1020, 4th Quart., 2012.

[22] E. Stinson and J. C. Mitchell, "Towards systematic evaluation of the evadability of bot/botnet detection methods," in *Proc. WOOT*, vol. 8, 2008, pp. 1–9.

[23] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. NDSS*, vol. 9, 2009, pp. 1–14.

[24] I. Homoliak, M. Teknos, M. Ochoa, D. Breitenbacher, S. Hosseini, and P. Hanacek, "Improving network intrusion detection classifiers by non-payload-based exploit-independent obfuscations: An adversarial approach," 2018, *arXiv:1805.02684*. [Online]. Available: http://arxiv.org/abs/1805.02684

[25] M. J. Hashemi, G. Cusack, and E. Keller, "Towards evaluation of NIDSs in adversarial setting," in *Proc. 3rd ACM CoNEXT Workshop Big Data, Mach. Learn. Artif. Intell. Data Commun. Netw. (Big-DAMA)*, 2019, pp. 14–21.

[26] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," 2016, *arXiv:1611.02770*. [Online]. Available: http://arxiv.org/abs/1611.02770

[27] I. Corona, G. Giacinto, and F. Roli, "Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues," *Inf. Sci.*, vol. 239, pp. 201–225, Aug. 2013.

[28] D. J. Chaboya, R. A. Raines, R. O. Baldwin, and B. E. Mullins, "Network intrusion detection: Automated and manual methods prone to attack and evasion," *IEEE Security Privacy*, vol. 4, no. 6, pp. 36–43, 2006.

[29] G. Vigna, W. Robertson, and D. Balzarotti, "Testing network-based intrusion detection signatures using mutant exploits," in *Proc. 11th ACM Conf. Comput. Commun. Secur. (CCS)*, 2004, pp. 21–30.

[30] D. Mutz, G. Vigna, and R. Kemmerer, "An experience developing an IDS stimulator for the black-box testing of network intrusion detection systems," in *Proc. 19th Annu. Comput. Secur. Appl. Conf.*, 2003, pp. 374–383.

[31] K. M. Tan, K. S. Killourhy, and R. A. Maxion, "Undermining an anomaly-based intrusion detection system using common exploits," in *Proc. Int. Symp. Recent Adv. Intrusion Detection (RAID)*. Berlin, Germany: Springer, 2002, pp. 54–73.

[32] H. G. Kayacik and A. N. Zincir-Heywood, "Mimicry attacks demystified: What can attackers do to evade detection?" in *Proc. 6th Annu. Conf. Privacy, Secur. Trust*, Oct. 2008, pp. 213–223.

[33] H. G. Kayacik, A. N. Zincir-Heywood, M. I. Heywood, and S. Burschka, "Generating mimicry attacks using genetic programming: A benchmarking study," in *Proc. IEEE Symp. Comput. Intell. Cyber Secur.*, Mar. 2009, pp. 136–143.

[34] P. Fogla, M. I. Sharif, R. Perdisci, O. M. Kolesnikov, and W. Lee, "Polymorphic blending attacks," in *Proc. USENIX Secur. Symp.*, 2006, pp. 1–16.

[35] P. Fogla and W. Lee, "Evading network anomaly detection systems: Formal reasoning and practical techniques," in *Proc. 13th ACM Conf. Comput. Commun. Secur. (CCS)*, 2006, pp. 59–68.

[36] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2574–2582.

[37] X. Yuan, P. He, Q. Zhu, and X. Li, "Adversarial examples: Attacks and defenses for deep learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 9, pp. 2805–2824, Sep. 2019.

[38] G. Chen *et al.*, "Who is real bob? Adversarial attacks on speaker recognition systems," 2019, *arXiv:1911.01840*. [Online]. Available: http://arxiv.org/abs/1911.01840

[39] Y. Zang *et al.*, "Word-level textual adversarial attacking as combinatorial optimization," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 6066–6080.

[40] Y. Wang, Y.-A. Tan, W. Zhang, Y. Zhao, and X. Kuang, "An adversarial attack on DNN-based black-box object detectors," *J. Netw. Comput. Appl.*, vol. 161, Jul. 2020, Art. no. 102634.

[41] Z. Wang, "Deep learning-based intrusion detection with adversaries," *IEEE Access*, vol. 6, pp. 38367–38384, 2018.

[42] J. Clements, Y. Yang, A. Sharma, H. Hu, and Y. Lao, "Rallying adversarial techniques against deep learning for network security," 2019, *arXiv:1903.11688*. [Online]. Available: https://arxiv.org/abs/1903.11688

[43] O. Ibitoye, O. Shafiq, and A. Matrawy, "Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks," 2019, *arXiv:1905.05137*. [Online]. Available: http://arxiv.org/abs/1905.05137

[44] A. Piplai, S. S. L. Chukkapalli, and A. Joshi, "NAttack! Adversarial attacks to bypass a GAN based classifier trained to detect network intrusion," 2020, *arXiv:2002.08527*. [Online]. Available: http://arxiv.org/abs/2002.08527

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.

[45] Z. Lin, Y. Shi, and Z. Xue, "IDSGAN: Generative adversarial networks for attack generation against intrusion detection," 2018, *arXiv:1809.02077*. [Online]. Available: http://arxiv.org/abs/1809.02077
[46] X. Peng, W. Huang, and Z. Shi, "Adversarial attack against DoS intrusion detection: An improved boundary-based method," in *Proc. IEEE 31st Int. Conf. Tools With Artif. Intell. (ICTAI)*, Nov. 2019, pp. 1288–1295.
[47] G. Apruzzese, M. Colajanni, and M. Marchetti, "Evaluating the effectiveness of adversarial attacks against botnet detectors," in *Proc. IEEE 18th Int. Symp. Netw. Comput. Appl. (NCA)*, Sep. 2019, pp. 1–8.
[48] M. F. Umer, M. Sher, and Y. Bi, "Flow-based intrusion detection: Techniques and challenges," *Comput. Secur.*, vol. 70, pp. 238–254, Sep. 2017.
[49] J. J. Davis and A. J. Clark, "Data preprocessing for anomaly based network intrusion detection: A review," *Comput. Secur.*, vol. 30, nos. 6–7, pp. 353–375, Sep. 2011.
[50] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2016, pp. 407–414.
[51] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Jul. 2009, pp. 1–6.
[52] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2014, pp. 2672–2680.
[53] R. Poli, J. Kennedy, and T. Blackwell, "Particle swarm optimization," *Swarm Intell.*, vol. 1, no. 1, pp. 33–57, 2007.
[54] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, Jan. 2003.
[55] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.
[56] A. Qayyum, M. H. Islam, and M. Jamil, "Taxonomy of statistical based anomaly detection techniques for intrusion detection," in *Proc. IEEE Symp. Emerg. Technol.*, Sep. 2005, pp. 270–276.
[57] A. Sperotto, G. Schaffrath, R. Sadre, C. Morariu, A. Pras, and B. Stiller, "An overview of IP flow-based intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 12, no. 3, pp. 343–356, 3rd Quart., 2010.
[58] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, Apr. 2017, pp. 506–519.
[59] S. Mrdovic and B. Perunicic, "Kerckhoffs' principle for intrusion detection," in *Proc. Netw.-13th Int. Telecommun. Netw. Strategy Planning Symp.* IEEE, 2008, pp. 1–8.
[60] R. C. Eberhart and Y. Shi, "Comparing inertia weights and constriction factors in particle swarm optimization," in *Proc. Congr. Evol. Comput. (CEC)*, vol. 1, 2000, pp. 84–88.



Dongqi Han received the B.S. degree in computer science from Jilin University, China. He is currently pursuing the Ph.D. degree with the Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include network anomaly detection, network security, and machine learning security.



Zhiliang Wang (Member, IEEE) received the B.E., M.E., and Ph.D. degrees in computer science from Tsinghua University, China, in 2001, 2003, and 2006, respectively. He is currently an Associate Professor with the Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include formal methods and protocol testing, next generation Internet, network measurement, and network security.



Ying Zhong received the master's degree from the College of Information Science and Engineering, Hunan University, China. He is currently pursuing the Ph.D. degree with the Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include the machine learning, big data, network security, data analysis, and mining algorithms and their parallel implementation.



Wenqi Chen received the B.E. degree in computer science from Tsinghua University, China. He is currently pursuing the Ph.D. degree with the Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include cyberspace security and network intrusion detection.



Jiahai Yang (Senior Member, IEEE) received the M.S. and Ph.D. degrees in computer science from Tsinghua University, Beijing, China, in 1992 and 2003, respectively. He is currently a Professor with Tsinghua University. His research interests include Internet architecture and its protocols, IP routing technology, network measurement, network management, cloud computing, and big data.



Shuqiang Lu is currently pursuing the Ph.D. degree with the Department of Computer Science and Technology, Tsinghua University. His research interests include machine learning and security.



Xingang Shi (Member, IEEE) received the B.S. degree from Tsinghua University and the Ph.D. degree from The Chinese University of Hong Kong. He is currently working with the Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include network measurement and routing protocols.



Xia Yin (Senior Member, IEEE) received the B.E., M.E., and Ph.D. degrees in computer science from Tsinghua University, in 1995, 1997, and 2000, respectively. She is currently a Full Professor with the Department of Computer Science and Technology, Tsinghua University. Her research interests include future Internet architectures, formal methods, protocol testing, and largescale Internet routing.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 12:14:10 UTC from IEEE Xplore. Restrictions apply.